Assignment 2 · Parallel & Distributed Computing

Muhammad Ayaan Naveed · 23i-0041 · CS-D
Sparse Matrices, Graph Analytics and Stencil Computation using OpenMP and SIMD

1. Machine, tools and measurement protocol

All experiments were executed in the same Linux cloud VM. Measurements describe this machine, not a personal laptop. Timings are kernel wall times from omp_get_wtime(); reading, allocation, printing and initialisation are excluded. Each executable takes the minimum of five runs. Task 3 times 50 SpMVs per run and reports the smallest of five 50-run means. No speedups or validation values are simulated.

Property	Observed configuration
CPU	AMD EPYC 9V74 80-Core Processor (KVM virtual machine)
Available processors	5 logical CPUs, reported as 5 cores / 1 thread per core; cgroup CPU quota 4 cores
Memory / NUMA	32 GiB cgroup limit; one exposed NUMA node
Caches (hypervisor report)	L1d 96 KiB / 3 instances, L1i 96 KiB / 3; L2 3 MiB / 3; L3 32 MiB shared
Instruction support	AVX, AVX2, FMA and AVX-512F are present
Register widths	AVX/AVX2: 256 bits = 8 float / 4 double lanes; AVX-512: 512 bits = 16 float / 8 double
Compiler / library	GCC 14.2.0; SuiteSparse:GraphBLAS 7.4.0, GraphBLAS C API 2.0
Build settings	-O2 -std=c11 -fopenmp -mavx2 -mfma -ffp-contract=off
Thread settings	1, 2, 4; OMP_DYNAMIC=false, OMP_PROC_BIND=true, OMP_PLACES=cores, OMP_WAIT_POLICY=PASSIVE

Eight threads exceed the five visible hardware threads and the four-core quota, so 8-thread tests are excluded. Four threads are the highest useful prescribed count. AVX-512 is deliberately unused so that all tasks share the prescribed AVX2/FMA target. Scalar kernels explicitly disable automatic vectorisation; SIMD variants are identified in code. Explicit AVX2 FMA intrinsics in Task 3 remain fused even with automatic contraction disabled.

Data provenance and limits

The Stanford endpoint was blocked. Two independent accessible GitHub copies of web-Google.txt were checked; their canonical LF text matches exactly. The graph has 875,713 distinct endpoint IDs and 5,105,039 directed edges. Vertex IDs are compacted in sorted order and retained for output. No dense graph matrix is ever allocated. The full dataset and all binaries are kept outside the submission.

```
Canonical SHA256 (LF text):
543b17f820d78af6616618f3ece2da836e917f089140d68113045313a3523d45
```

Mirror pinned by the reproduction script: bhargavikesarkar/parallel-Webgraph-analysis, commit f7ac0ebb93c25e42ef99da7b8685259f74a7598b. A second copy in sanya-sajin/Google_Graph_Analysis matched after CRLF normalisation. VM contention and quota throttling can affect scaling; the minimum of five reduces but does not eliminate this noise.

2. Task 1 · Correctness, sharing and dependence

N = 2^24 float inputs cover [−π, π), with x[i] = −π + 2πi/N and sinf as the reference. Listing 2 performs j = 1 … terms corrections after the initial x. The update denom *= (2*j+2)*(2*j+3) overflows when forming 13! after j=5; the declaration int denom = 6 selects an insufficient signed 32-bit type. The minimal fix is float denom = 6.0f. The supplied program also models the original denominator using defined uint32_t wrapping; actual signed overflow in the original C is undefined, so its exact output is not portable.

Corrections	Original wrap-model max |error|	Corrected max |error|
4	0.00692555854971	0.00692555854971
8	0.993562610941	5.75557351112e-07
12	927.854370205	5.55068254471e-07

The four-correction error is truncation error. Increasing to eight corrections sharply reduces it; beyond this, single-precision rounding sets an error floor of roughly 10^{-6}. Tiny changes at twelve corrections do not imply further factorial overflow. All measured thread, SIMD and hybrid versions match their corrected scalar output bit for bit on this input and build; the executable additionally prints maximum absolute differences and bit-difference counts.

Variable	Sharing	Reason
N, terms	shared	Read-only loop bounds / correction count
x, y	shared	Shared array storage; x is read-only and each i owns y[i]
i	private	OpenMP associated-loop induction variable
j	private	Local inner-loop induction variable for one input
value	private	Per-input accumulator
numer	private	Per-input power recurrence
denom	private	Per-input factorial recurrence
sign	private	Per-input alternating-sign recurrence

Why the outer loop is parallel, but not the inner recurrence

Each outer iteration reads x[i] and writes a unique y[i]. No iteration needs another output. Inner iteration j reads value, numer, denom and sign produced by j−1; all four carry loop dependence. Parallelising that inner loop directly would break the recurrence. Instead, SIMD lanes each evaluate a different input while preserving its internal correction order.

Compiler vectorisation evidence

```
23i-0041-CS-D-TASK1-V1.c:68:22: optimized: loop vectorized using 32 byte vectors
23i-0041-CS-D-TASK1-V1.c:68:22: optimized: loop vectorized using 16 byte vectors
23i-0041-CS-D-TASK1-V1.c:68:35: optimized: loop vectorized using 32 byte vectors
23i-0041-CS-D-TASK1-V1.c:68:35: optimized: loop vectorized using 16 byte vectors
```

Fixed-count 4/8/12-correction specialisations unroll the recurrence before vectorising across inputs. The one-thread SIMD loop uses omp simd; the hybrid loop uses omp parallel for simd with runtime scheduling. The compiler report above concerns the actual output loops, not merely initialisation. Automatic FMA contraction is disabled to keep scalar and SIMD operation ordering comparable.

3. Task 1 · Performance and scheduling

Variant	Threads	Seconds	Speedup	Efficiency
scalar	1	0.254906	1.000	1.000
threads	1	0.246939	1.032	1.032
simd	1	0.024102	10.576	10.576
hybrid	1	0.024404	10.445	10.445
threads	2	0.126209	2.020	1.010
hybrid	2	0.012923	19.725	9.862
threads	4	0.065758	3.876	0.969
hybrid	4	0.007386	34.514	8.629

Speedup = scalar sequential time / variant time; efficiency = speedup / executing thread count. Tables use the one-thread scalar measurement as their common reference. Hybrid efficiency can exceed one because it includes SIMD and specialisation gains. One-thread SIMD efficiency is therefore equal to its SIMD speedup. The sequential and SIMD-only rows appear once; two- and four-thread rows apply only to parallel variants. The same convention is used in later tables.

Highest-thread-count scheduling comparison

Schedule	Threads seconds	Hybrid seconds
static	0.065758101	0.007385541
dynamic,1	0.896050232	0.797897068
dynamic,4096	0.064314404	0.006909858

The fastest measured variant is hybrid on 4 thread(s), at 0.007386 s. Every input performs the same number of corrections, so there is little load imbalance for dynamic scheduling to repair. Static assignment gives large contiguous ranges and little work-distribution overhead. Dynamic,1 invokes the scheduling machinery for individual elements and fragments SIMD chunks; dynamic,4096 amortises that overhead over larger blocks.

The Taylor recurrence contains serial dependency chains within each input. SIMD exposes independent chains across inputs, but a pragma alone does not guarantee a speedup: instruction throughput and division latency matter. Fixed-count unrolling also lets GCC constant-fold factorial denominators; any speedup includes this optimisation and removed loop/call overhead, not just vector width. Thread scaling is limited by the four-core quota and by the VM scheduler. Reported values should be interpreted as measurements of this compiler and processor rather than an assumption that all four variants must become faster.

Reproducible command examples

```
make BUILD=/tmp/pdc-build all
/tmp/pdc-build/task1 16777216 1 12 static
/tmp/pdc-build/task1 16777216 2 12 static
/tmp/pdc-build/task1 16777216 4 12 static
/tmp/pdc-build/task1 16777216 4 12 dynamic1
/tmp/pdc-build/task1 16777216 4 12 dynamic4096
```

The program runs all three correctness correction counts before timing each variant. Allocation, reference calculation and error checking occur outside measured intervals. The full reproduction driver sets affinity and passive OpenMP waiting consistently for every task.

4. Task 2 · Sparse PageRank results

For each edge src → dst, incoming CSR row dst stores column src and value 1/outdeg(src). Construction counts row lengths, prefix-sums them to row_ptr, and scatters edges with a cursor per row. It requires O(n + nnz) storage. Every iteration uses d = 0.85, sums dangling rank, applies M to r, and stops at L1 change < 10^{-8}, or after 200 iterations. Rank buffers are swapped by pointer; only final extraction may copy a buffer after timing.

G6 vertex	Measured rank	Assignment (rounded)
0	0.186037070352	0.186037
1	0.104065755368	0.104066
2	0.273216198263	0.273216
3	0.141116884995	0.141117
4	0.189869538403	0.189870
5	0.105694552619	0.105695

G6 converges in 44 iterations; rank mass is 1.000000000000. Its ranks pass the stated tolerance. For the full Google graph: n = 875713, nnz = 5105039, dangling pages = 136259; convergence takes 90 iterations, and rank mass is 1.000000000001.

Position	Original page ID	Rank
1	597621	0.000923724948
2	41909	0.000921131246
3	163075	0.000904004426
4	537039	0.000898831806
5	384666	0.000786892445
6	504140	0.000765116054
7	486980	0.000724940304
8	605856	0.000717955263
9	32163	0.000712571850
10	558791	0.000709185934

Dangling-page experiment

With the dangling term deliberately omitted, final rank mass becomes 0.697562227838 after 89 iterations. This version is an experiment, not the submitted correct algorithm. Rank leaks through pages with zero out-degree because their columns contain no transition probability. Uniform redistribution d·s/n preserves total probability. G6 has no dangling vertices, so it alone cannot expose this bug; the Google experiment is necessary.

5. Task 2 · Thread counts, scheduling and correctness

Threads	Schedule	Seconds	Speedup	Efficiency
1	static	1.457331	1.062	1.062
1	dynamic1024	1.500624	1.032	1.032
1	guided	1.517093	1.020	1.020
2	static	0.795012	1.947	0.974
2	dynamic1024	0.807892	1.916	0.958
2	guided	0.753317	2.055	1.027
4	static	0.443396	3.491	0.873
4	dynamic1024	0.410249	3.773	0.943
4	guided	0.424153	3.650	0.912

The common scalar reference for this table is 1.547967 s (minimum of five scalar runs). Each process also records its own scalar time in the raw outputs, so measurement noise can be inspected independently.

Incoming row lengths have mean 5.829580, standard deviation 39.228970, and maximum 6326. This distribution is highly irregular: a few popular pages have many incoming links, while many rows are short or empty. Equal row counts are therefore not equal work. Dynamic chunks can reduce imbalance; guided scheduling gradually shrinks chunks, reducing scheduler overhead while allowing redistribution near the end.

The fastest measured policy/count is dynamic1024 with 4 thread(s) at 0.410249 s. This is the empirical choice for this VM. Dynamic,1024 trades balancing against work-distribution and locality costs; guided trades progressively smaller chunks against similar costs. Static can still win when scheduling overhead, CSR locality or bandwidth dominates the cost of uneven rows. One should choose from measurements, not from irregularity alone.

Race freedom and reproducibility

Every SpMV row writes one b[i], so no row shares an output. Current rank a is read-only throughout the sweep. Dangling mass and L1 change use OpenMP sum reductions. The barrier at the end of the sweep completes all outputs before the rank pointers are exchanged. Different reduction trees can change low-order rounding; the maximum final rank difference from the sequential baseline is checked against 10^{-10}.

Largest observed final-rank difference over all Google scheduling runs: 6.83e-18. Input lines beginning with # are ignored; endpoint IDs are sorted and uniqued, then binary-searched for relabelling. The source/destination direction is preserved. Top-ten output uses the reverse mapping, so the page identifiers are original IDs rather than CSR row numbers.

```
/tmp/pdc-build/task2 /path/web-Google.txt 4 static
/tmp/pdc-build/task2 /path/web-Google.txt 4 dynamic1024
/tmp/pdc-build/task2 /path/web-Google.txt 4 guided
/tmp/pdc-build/task2 /path/web-Google.txt 4 static 1  # experiment only
```

6. Task 3 · B in COO, CSR and CSC

B is stored as sparse arrays throughout. The following diagrams list every entry, using four-byte indices and four-byte float values. Array payload sizes exclude constant-size C structure metadata and allocator bookkeeping.

COO · 12·nnz = 156 bytes

```
position 0 1 2 3 4 5 6 7 8 9 10 11 12
row      0 0 1 1 2 2 2 3 4 4 4 5 5
col      0 1 1 4 0 2 3 3 2 4 5 3 5
val      4 1 4 9 6 4 5 4 8 4 1 6 4
```

CSR · 8·nnz + 4·(n+1) = 132 bytes

```
row_ptr  0 2 4 7 8 11 13
col      0 1 1 4 0 2 3 3 2 4 5 3 5
val      4 1 4 9 6 4 5 4 8 4 1 6 4
```

Conversion CSR → COO expands each interval [row_ptr[i], row_ptr[i+1]) into explicit row index i; col and val are copied to identical positions. Rows own disjoint half-open intervals because row_ptr is non-decreasing. Consequently, parallel rows never write the same COO element, even for empty rows.

CSC · 8·nnz + 4·(n+1) = 132 bytes

```
col_ptr  0 2 4 6 9 11 13
row_idx  0 2 0 1 2 4 2 3 5 1 4 4 5
val      4 6 1 4 4 8 5 4 6 9 4 1 4
```

CSR groups each output dot product by row. CSC groups nonzeros by input column; it is useful for column access but a straightforward column-parallel Ax would have multiple threads adding into the same y[i]. The CSC representation is shown for the required diagram; no CSC kernel is required.

A and B correctness

```
VALID matrix=A kernel=0 y=15,28,50,28
VALID matrix=A kernel=1 y=15,28,50,28
VALID matrix=A kernel=2 y=15,28,50,28
VALID matrix=B kernel=0 y=6,53,38,16,50,48
VALID matrix=B kernel=1 y=6,53,38,16,50,48
VALID matrix=B kernel=2 y=6,53,38,16,50,48
```

All three kernels reproduce the required outputs exactly on these integer-valued checks. The small sizes exercise the ELL scalar tail and DIA boundary handling; the million-row Laplacian additionally exercises full AVX2 blocks. Conversion to COO is checked entry-by-entry. DIA is also tested with a one-byte limit and must reject without allocating output tables.

7. Task 3 · ELL and DIA conversion diagrams

CSR → ELL · W = 3 · 8·n·W = 144 bytes

Find W with reduction(max:W), then fill one independent row per thread. Each slot is a contiguous column-major group of rows, index k·n+i. A dot denotes zero padding; the actual stored padding is value 0, column 0.

Slot k	col[k·n ... k·n+5]	val[k·n ... k·n+5]
0	0 1 0 3 2 3	4 4 6 4 8 6
1	1 4 2 0 4 5	1 9 4 0 4 4
2	0 0 3 0 5 0	0 0 5 0 1 0

AVX2 gather loads x[col] before multiplication. A zero matrix value does not suppress an invalid memory access, so padded col must be a valid index. Column zero is safe for every nonempty matrix.

CSR → DIA · offsets sorted increasingly

Diagonal / offset	val[d·n ... d·n+5]
0 / -2	0 0 6 0 8 6
1 / 0	4 4 4 4 4 4
2 / 1	1 0 5 0 1 0
3 / 3	0 9 0 0 0 0

DIA needs nd = 4 diagonals and 4·nd·(n+1) = 112 bytes. The conversion first flags offsets col−row+n−1 in a 2n−1 scratch array, numbers occupied flags in increasing order, then fills each row’s entries in parallel. Simultaneous ordinary writes of 1 to the same flag are a data race, despite equal values. This implementation uses omp atomic write. After the flagging barrier, serial numbering is safe; different output rows write different positions within each diagonal.

The max_bytes guard is evaluated after counting occupied offsets and before allocating the DIA offsets/value tables; temporary scratch flags are freed on rejection. Dividing max_bytes by 4·(n+1) avoids a multiplication overflow in the size test. Input CSR must not contain duplicate coordinates for this conversion; the generated Laplacian and A/B satisfy that precondition.

Vector kernels and output ownership

ELL gives blocks of eight rows to OpenMP. Every lane owns one output row; loadu_si256 loads column indices, i32gather_ps fetches x, loadu_ps reads values, fmadd_ps accumulates, and storeu_ps writes results. Remaining n mod 8 rows use a scalar FMA tail. DIA likewise distributes row blocks to threads, then traverses diagonals within each block; contiguous loads replace gathers. Boundary fragments are handled scalarly. Parallelising the diagonal loop with a shared y would race because all diagonals contribute to the same output rows.

8. Task 3 · Storage, kernels and format choice

Matrix	n / nnz	CSR bytes	ELL bytes	DIA bytes
L (1024^2)	1048576 / 5238784	46104580	41943040	20971540
Google M (float)	875713 / 5105039	44343168	44318083504	5120541455064

L has W = 5, nd = 5; G has W = 6326, nd = 1461819. Google ELL/DIA sizes are computed from incoming CSR row lengths and occupied offsets only; neither huge format is allocated. The CSR size here uses float values, whereas Task 2 stores double weights.

Variant	Threads	Seconds	Speedup	Efficiency
scalar	1	0.003878	1.000	1.000
threads	1	0.003898	0.995	0.995
ell_simd	1	0.002898	1.338	1.338
ell_hybrid	1	0.002762	1.404	1.404
dia_simd	1	0.001577	2.458	2.458
dia_hybrid	1	0.001589	2.440	2.440
threads	2	0.002177	1.782	0.891
ell_hybrid	2	0.001488	2.606	1.303
dia_hybrid	2	0.000908	4.273	2.136
threads	4	0.001387	2.796	0.699
ell_hybrid	4	0.001347	2.879	0.720
dia_hybrid	4	0.000680	5.704	1.426

Fastest measured kernel: dia_hybrid, 4 thread(s), 0.000680 s per SpMV. Maximum observed difference from the scalar reference is 2.98e-08; the limit checked by the executable is 2×10^{-6}. FMA rounds a product-plus-sum once; scalar multiplication/addition rounds twice. DIA boundary fragments can also alter accumulation grouping. These finite-precision effects explain the small non-bit-identical differences.

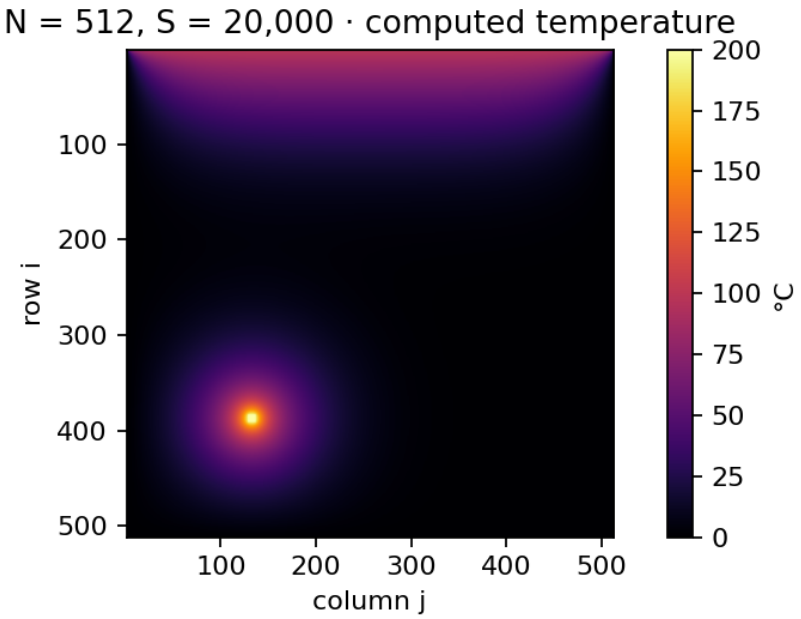
Hybrid efficiency can exceed one because speedup includes SIMD as well as thread-level acceleration; it is not a fraction of total CPU utilisation. The Laplacian is banded with five occupied offsets and at most five entries per row: ELL padding is modest, and DIA avoids per-entry column indices and gathers. Google has long irregular rows and widely spread diagonals. One long row forces every ELL row to that width; DIA allocates n floats for almost every offset. CSR therefore fits the graph, whereas DIA is structurally well suited to L. Memory bandwidth, gather cost and the selected CPU quota determine the measured ordering within those structural choices.

9. Task 4 · Heat validation and final field

Two contiguous $(N+2)^2$ double buffers hold old and new temperatures. The top boundary is 100°C, other edges 0°C; an 8×8 source beginning at (3N/4, N/4) is fixed at 200°C in both buffers. Every other interior cell starts at zero. Updated cells use 0.2 times the sum of their old value and four old neighbours. Source rows are split into segments outside the source, so SIMD column loops contain no source test.

Variant / threads	Centre	Mean	Final δ
scalar / 1	2.455707195462	18.19280078449	0.001203781436356
threads / 1	2.455707195462	18.19280078449	0.001203781436356
threads / 2	2.455707195462	18.19280078449	0.001203781436356
threads / 4	2.455707195462	18.19280078449	0.001203781436356
persistent / 4	2.455707195462	18.19280078449	0.001203781436356

All displayed values agree to at least ten significant digits, and round to the assignment's centre = 2.4557, mean = 18.1928 and δ = 1.20×10^{-3}. The mean includes source cells; δ excludes fixed cells and measures only the final step, not the largest change over all time.



A single in-place array would mix old and already updated values: the result becomes sweep-order dependent (Gauss–Seidel-like), and parallel reads/writes introduce races. Separate arrays and a barrier before pointer exchange retain Jacobi semantics. OpenMP v1 opens a parallel region each step; v2 opens one persistent team, uses an omp for max reduction, and swaps pointers in omp single with its implicit barrier.

10. Task 4 · Performance and memory behaviour

Variant	Threads	$512^2 \times 20{,}000$: s / speedup / eff.	$2048^2 \times 1{,}000$: s / speedup / eff.
scalar	1	5.14713 / 1.000 / 1.000	5.61913 / 1.000 / 1.000
simd	1	1.96133 / 2.624 / 2.624	4.25561 / 1.320 / 1.320
threads	1	5.27550 / 0.976 / 0.976	5.72372 / 0.982 / 0.982
hybrid	1	1.97774 / 2.603 / 2.603	4.32151 / 1.300 / 1.300
persistent	1	2.12903 / 2.418 / 2.418	4.14360 / 1.356 / 1.356
threads	2	3.72526 / 1.382 / 0.691	3.02982 / 1.855 / 0.927
hybrid	2	1.60125 / 3.214 / 1.607	2.50458 / 2.244 / 1.122
persistent	2	1.68517 / 3.054 / 1.527	2.47651 / 2.269 / 1.134
threads	4	3.02837 / 1.700 / 0.425	2.13009 / 2.638 / 0.659
hybrid	4	1.64695 / 3.125 / 0.781	1.43912 / 3.905 / 0.976
persistent	4	1.93807 / 2.656 / 0.664	1.45346 / 3.866 / 0.967

Experiment	Seconds	Interpretation
collapse(2), N=512	3.472066	4 threads; flattened iteration space, source branch
collapse(2), N=2048	2.496995	4 threads; flattened iteration space, source branch
swapped loops, N=2048	9.259518	4 threads; column-major traversal of row-major arrays
$4096^2 \times 200$, init=0	1.358221181	initialisation separately: 0.014220 s
$4096^2 \times 200$, init=1	1.303274742	initialisation separately: 0.008882 s

N=512: fastest measured variant is hybrid with 2 thread(s), 1.601251 s. Persistent/SIMD v2 versus scalar-loop v1 combines vectorisation with team reuse; compare hybrid v1 with persistent v2 to isolate the team-lifetime change.

N=2048: fastest measured variant is hybrid with 4 thread(s), 1.439125 s. Persistent/SIMD v2 versus scalar-loop v1 combines vectorisation with team reuse; compare hybrid v1 with persistent v2 to isolate the team-lifetime change.

Persistent teams remove repeated region entry, but v2 also has barriers for the delta reset, reduction and pointer exchange. In these measurements, passive waiting and those extra barriers can outweigh the saved region overhead, especially on the small grid. A large grid makes stencil traffic dominate. collapse(2) can expose more iterations, but these grids already offer hundreds or thousands of rows; flattening adds index arithmetic and hinders the branch-free row-segment vector loop. Swapped loops advance by a whole row, wasting cache-line locality and increasing TLB/cache pressure. In row-major traversal, adjacent columns reuse contiguous cache lines.

First touch assigns physical pages on the first write. Parallel initialisation uses the sweep’s static row schedule; serial initialisation first-touches all pages on one thread. This VM exposes one NUMA node, so there is no demonstrated cross-socket placement advantage. Initialisation time and subsequent kernel time are reported separately; small differences can reflect caches, paging and VM noise rather than NUMA migration.

11. Task 5 · GraphBLAS and BFS

The C library was installed locally by downloading Debian libgraphblas-dev and libgraphblas7 7.4.0+dfsg-2 through APT with repository signature and package-hash verification, then extracting under /workspace/pdc-tools/graphblas. No Python GraphBLAS fallback was needed. The terminal panel below renders the recorded successful compile/run transcript, including the API version.

```
$ gcc -O2 -std=c11 -fopenmp -mavx2 -mfma -ffp-contract=off -I/workspace/pdc-tools/graphblas/usr/include
Task5/23i-0041-CS-D-TASK5-V1.c -o /workspace/pdc-final-build/gb-example -L/workspace/pdc-
tools/graphblas/usr/lib/x86_64-linux-gnu -Wl,-rpath,/workspace/pdc-tools/graphblas/usr/lib/x86_64-linux-gnu
-lgraphblas -lm

$ /workspace/pdc-final-build/gb-example g6 4 0
GraphBLAS C API version: 2.0 ; SuiteSparse implementation 7.4.0
Vertices one hop from 0: 1 2
OUTDEGREE 2 1 2 1 2 2
TASK5 n=6 edges=10 threads=4 source=0 reached=6 maxlevel=4
RESULT variant=graphblas threads=4 seconds=0.000006039 validation=PASS
BFS 0 1 1 2 3 4
```

The G6 adjacency matrix stores A(src,dst)=true. One call to GrB_Matrix_reduce_Monoid with the plus monoid returns out-degrees (2, 1, 2, 1, 2, 2). The one-hop example returns vertices 1 and 2. G6 BFS from 0 returns levels (0, 1, 1, 2, 3, 4), matching the assignment.

Google BFS starts from Task 2’s highest-ranked original page 597621, reaches 600493 vertices, and has maximum level 31. Minimum of five BFS kernel times: 0.039571 s on four GraphBLAS threads. Every output level is independently checked against a scalar queue BFS.

Depth	Count	Depth	Count	Depth	Count
0	1	11	99935	22	40
1	22	12	55730	23	21
2	303	13	28260	24	5
3	1284	14	13604	25	14
4	2401	15	7320	26	4
5	2601	16	3181	27	4
6	6350	17	1336	28	3
7	25130	18	732	29	2
8	78201	19	401	30	3
9	134805	20	157	31	3
10	138577	21	63		

Each level uses assign_INT32(level, q, …) followed by vxm over the OR–AND semiring with GrB_DESC_RSC. The complemented structural level mask excludes visited vertices, including the source whose stored level is zero. Replace semantics clear stale frontier entries. Blocking mode and nvals synchronisation make timing include completed operations.

Answers. (i) With plus–times, GrB_mxv corresponds to Task 3’s y=Ax SpMV, regardless of the library’s hidden sparse format. (ii) Without the mask, G6’s cycles repeatedly regenerate visited vertices; the frontier need not empty and levels get overwritten. (iii) GraphBLAS provides concise, tested sparse kernels and semiring/mask reuse; a disadvantage is less direct control over storage, scheduling and fusion than hand-written PageRank, plus a library dependency.

12. Reproduction, packaging and conclusions

Build and run on Linux (AVX2/FMA required)

```
sudo apt-get update
sudo apt-get install build-essential libgraphblas-dev git python3
cd 23i-0041-CS-D
make BUILD=/tmp/pdc-build all
make BUILD=/tmp/pdc-build check
sh 23i-0041-CS-D-TASK0-V1.sh
```

The shell script obtains the pinned full graph, verifies its canonical checksum, builds all tasks and runs the experiment driver. BUILD, DATA and RESULTS may be set to writable directories outside the submission folder. For a non-system GraphBLAS installation, pass GB_PREFIX and GB_LIBDIR to make, e.g. GB_PREFIX=/workspace/pdc-tools/graphblas/usr. The linker embeds an rpath for that library directory.

Direct experiment / report commands

```
python3 23i-0041-CS-D-TASK0-V1.py \
  --build /tmp/pdc-build --graph /path/web-Google.txt \
  --output /tmp/pdc-results
# Optional PDF regeneration only (not required for C programs):
python3 -m pip install reportlab matplotlib pillow
python3 23i-0041-CS-D-TASK0-V2.py \
  /tmp/pdc-results /tmp/pdc-build /tmp/Report.pdf
```

To regenerate the terminal panel, record a successful task5 compile command and its G6 execution in RESULTS/graphblas-install.txt. Raw JSON/text results, compiler .vec reports, datasets, generated figures, fields and executables are intentionally outside the submission. The provided PDF embeds measured tables, the B conversion diagrams, final heatmap and terminal transcript. The benchmark driver stops on any nonzero validation status. Additional checks passed: ASan/UBSan sparse-format boundary/tail cases; nonconsecutive 64-bit graph IDs; and whole-field agreement of all seven heat variants at 64^2, 101 steps.

Submission layout

```
23i-0041-CS-D/
  Task1/23i-0041-CS-D-TASK1-V1.c
  Task2/23i-0041-CS-D-TASK2-V1.c
  Task3/23i-0041-CS-D-TASK3-V1.c
  Task4/23i-0041-CS-D-TASK4-V1.c
  Task5/23i-0041-CS-D-TASK5-V1.c
  23i-0041-CS-D-TASK0-V1.h       # shared input / memory helpers
  23i-0041-CS-D-TASK0-V1.py      # experiments
  23i-0041-CS-D-TASK0-V2.py      # report generation
  23i-0041-CS-D-TASK0-V1.sh      # reproduction entry point
  Makefile                      # required conventional filename
  23i-0041-CS-D-Report.pdf
```

The ZIP contains this single top-level folder, code/build instructions and this PDF only; no binaries, .exe files, graph datasets or result logs. TASK0 identifies shared/reproduction code, and Task1–Task5 each contain their task source. The assignment's explicit requirement for a single Makefile necessitates its conventional filename. All code has the required name, student ID and assignment number header.

Conclusions and references

Correctness requires fixing the serial baseline before parallelising, preserving independent output ownership, and validating floating-point differences. Sparse-format choice should follow matrix structure: DIA/ELL are natural candidates for the banded grid, while CSR handles irregular graph rows economically. SIMD, scheduling and persistent teams must be assessed with measurements rather than assumed gains. One-node virtualised hardware limits claims about NUMA and physical-core scaling.

References: supplied CS3006 PDC Assignment 2 (updated), Listings 1–7 and Figures 1–9; Stanford SNAP web-Google dataset (snap.stanford.edu/data/web-Google.html); OpenMP API specification (openmp.org/specifications); Intel Intrinsics Guide; SuiteSparse:GraphBLAS 7.4 C header/API and GraphBLAS C API 2.0. The original problem credits Stanford CS149, Maryland CMSC498X/818X, RWTH OpenMP exercises, and Bell–Garland sparse-format examples.